Operating Systems Technologies

(Tecnologias de Sistemas Operativos)

Modern Storage APIs

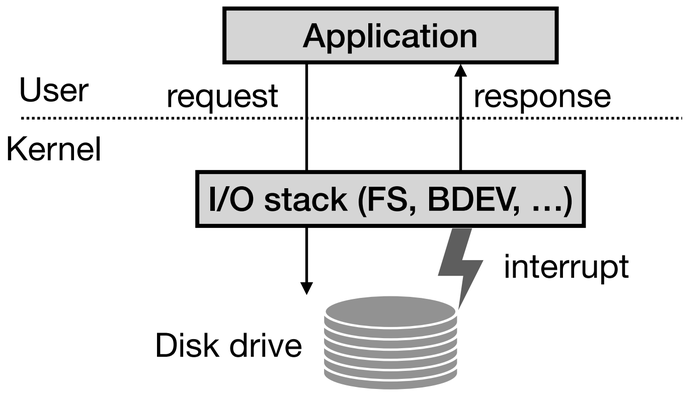
University of Minho

2025-2026

Motivation

Why change the POSIX API?

- **POSIX**
  - Traditional synchronous API that we have been using in the classes
  - *write() and read()* (and their variants) are blocking operations *i.e.*, block until the OS acknowledges handling the request
  - **Important:** writes can be acknowledged before they are on disk (as discussed in last class…)



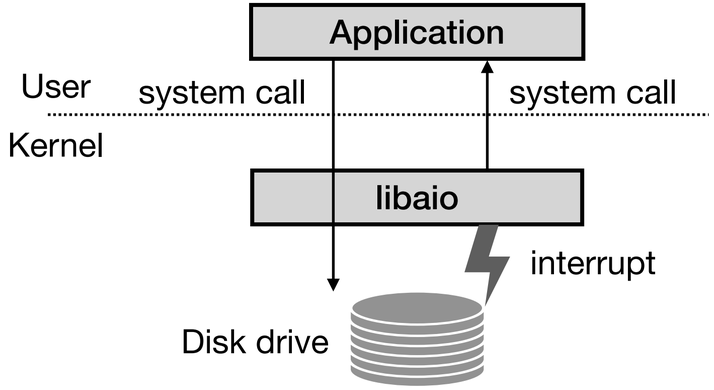
POSIX sketch

- But disks are getting faster (e.g., NVMe SSDs)…
  - Time spent by requests in kernel space is now considerable, affecting I/O latency/throughput
  - Extra memory copies between kernel and user spaces are costly performance-wise
  - Disk Interrupts lead to context switching and extra delay in receiving disk responses
- Is there an alternative approach?

libaio

Overview

- **The Linux Asynchronous API (libaio)**
  - Asynchronous API, *i.e.,* non-blocking requests
  - One system call to submit (*io_submit*) requests and another to later retrieve (*io_getevents*) the result of I/O requests
  - One can batch multiple operations under one submit syscall and let the OS/disk optimize (schedule, parallelize, merge) them
  - Interrupt-based completion at the device
  - Requires unbuffered access (*i.e.*, with O_DIRECT)
- Solves some of the problems identified for POSIX, but
  - Uses interrupts, kernel/user space memory copies, …
- Are there any other alternatives?



libaio sketch

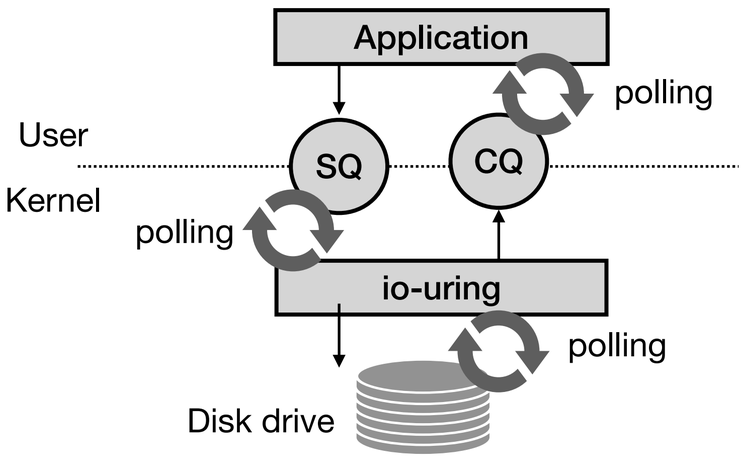
Modern APIs

IO_Uring and SPDK

- Let's hear from researchers working on modern high-performance I/O APIS!
- **Filipe Pereira**
  - The IO_Uring high-performance, asynchronous I/O interface for Linux
- **Rui Pedro Oliveira**
  - Storage Performance Development Kit (SPDK) for NVMe SSDs

IO_Uring

Overview

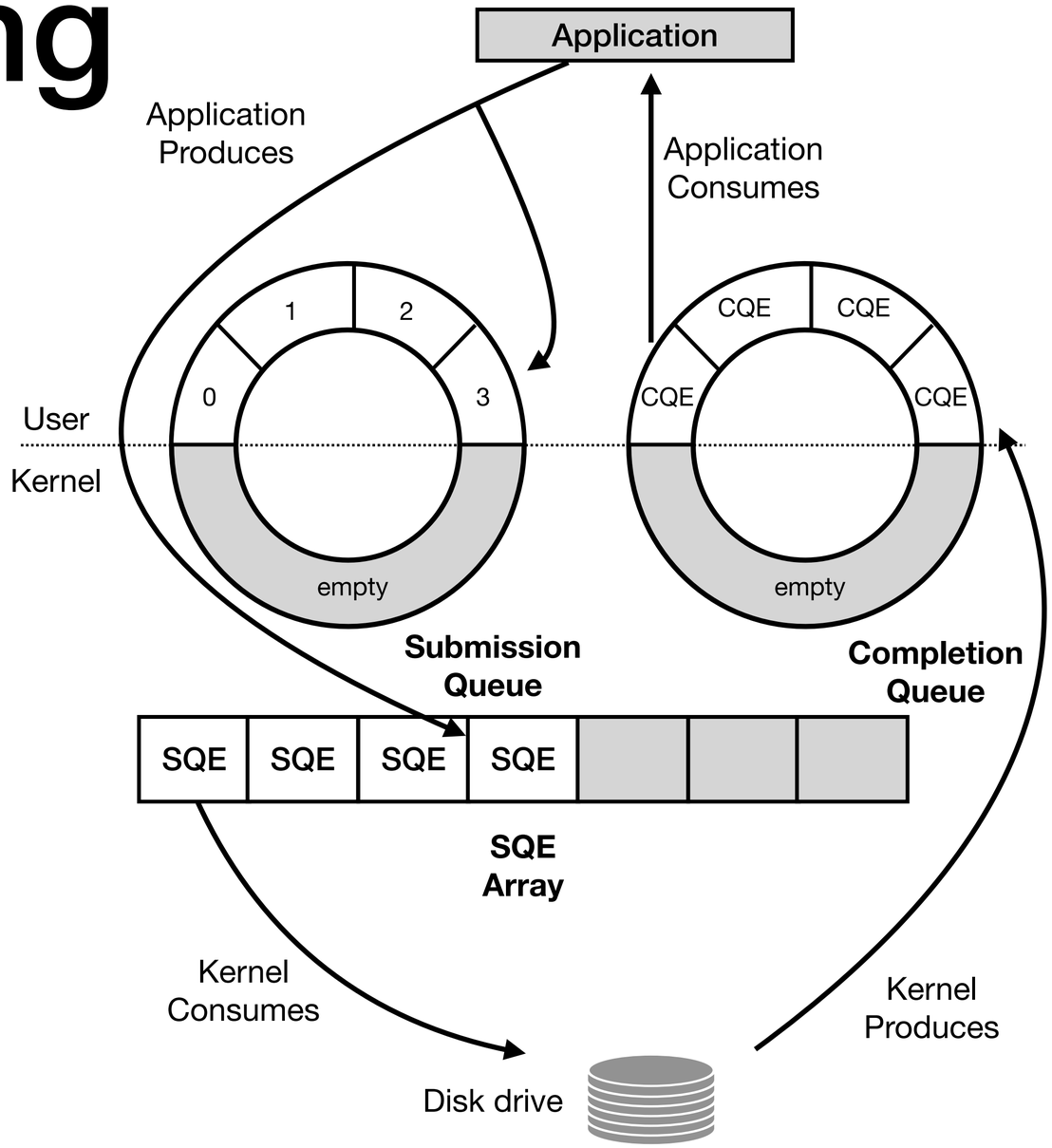


io-uring example with polling and zero syscalls

- Introduced in Linux 5.1 in 2019
- Designed to address the shortcomings of libaio
  - Didn't truly avoid blocking
  - High syscall and copy overhead
  - Very specific use cases (O_DIRECT needed)
- The application and the kernel communicate through two shared ring buffers (queues) in memory (one to submit requests, the other to receive responses)
- Both sides can read and write the ring buffers without crossing the user/kernel boundary on every operation
- **Advantages:** true async I/O ; batching; zero-copy; unified interface; multiple configurations

IO_Uring
Design

- Two ring buffer structures are memory-mapped into both user and kernel spaces
- **Submission Queue** (SQ):
  - Ring buffer of indices to the SQE Array, containing Submission Queue Entries (SQE)
  - The SQE describes the I/O request (operation type, file descriptor, buffer, …)
- **Completion Queue** (CQ)
  - Ring buffer of Completion Queue Events (CQE)
  - The CQE describes the I/O result (result, user_data, flags)
- I/O requests flow
  - Applications produce to SQ and consume from CQ
  - Kernel consumes from SQ and produces to CQ
  - A CQE always corresponds to a given SQE



IO_Uring

API overview

- Liburing is an abstraction over io_uring
  - Removes boilerplate code and provides simpler API helpers for standard use cases
- Create io_uring instance: **io_uring_queue_init()**
- Handle submissions:
  - Get an SQE: **io_uring_get_sqe()**
  - Prepare the operation: **io_uring_prep_read()** + **io_uring_sqe_set_data()**
  - Submit: **io_uring_submit()**
- Handle completions:
  - Block until a completion arrives: **io_uring_wait_cqe()**
  - Check the ring without blocking: **io_uring_peek_cqe()**
  - Get user_data: **io_uring_cqe_get_data()**
  - Mark as seen: **io_uring_cqe_seen()**

IO_Uring

Considerations - Queue depth

- Queue depth is the size of SQ
  - Submitting one request and then immediately waiting for it recreates the synchronous queue depth of 1
- The real impact on performance comes from submitting several independent requests at the same time
  - In other words, Io_uring is most interesting when the workload has many independent I/Os that can be executed in parallel

IO_Uring

Considerations - Tracking request state

- Completions can arrive in any order
- For most applications, knowing the requests that originated the completion is crucial
  - The SQE user_data field allows us to store literally anything there
  - The Kernel does not touch that field, it just moves it to the CQE

IO_Uring

Considerations - Ordering

- You submit 3 requests: w*rite A*, w*rite B*, f*sync*
  - What can go wrong?
- Besides CQEs arriving in any order, concurrent requests can also execute in any order
- The fsync can execute before the 2 writes, violating our persistence logic and corrupting the intended state
- Without ordering constraints, a lot of things can go wrong when dependent requests are issued in parallel

IO_Uring

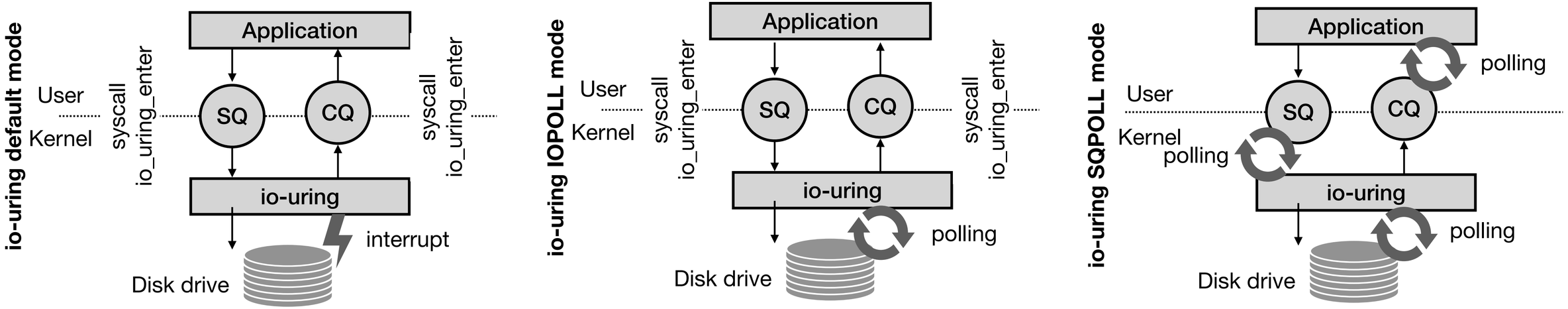
Considerations - Ordering

- **IOSQE_IO_LINK:** chain 2 consecutive SQEs
  - Set the flag on SQE N to link it to SQE N+1
  - SQE N+1 will not start until SQE N completes
  - Other independent SQEs or linked chains can run concurrently
  - If an SQE fails in the chain, subsequent ones are canceled
- **IOSQE_IO_DRAIN:** full ordering barrier
  - A drained SQE will not start until all previously submitted SQEs have completed
  - All SQEs submitted after it will wait for it to finish

IO_Uring

Considerations - Polling modes

- By **default**, the kernel uses interrupts for I/O completion and requires syscalls
- **IORING_SETUP_IOPOLL:** The kernel busy polls the device for completions
  - Can dramatically reduce latency on fast devices, but comes at the cost of burning cycles of a CPU core
- **IORING_SETUP_SQPOLL:** Dedicated kernel thread polls the SQ for new SQEs
  - Enables zero system calls, but the kernel thread consumes a full CPU core
  - The thread sleeps after an idle timeout



IO_Uring

Considerations - Performance

- Overall
  - Reduced syscall overhead
  - Zero copy submission/completion
  - Enable more parallelism on the device
- Comparison (2019)[1]
  - Random read workload
  - Libaio: 608k IOPS
  - IO_Uring: 1.2M IOPS (without polling); 1.7M IOPS (polling)

[1]Results taken from **Efficient IO with io_uring.** J. Axboe, Linux Kernel Documentation 2019

IO_Uring

Conclusion

- Good fit:
  - Many independent I/Os that we can execute concurrently
  - Latency critical paths where syscall and interrupt latency are measurable
  - I/O bound workloads in general
- Bad fit:
  - Many ordering dependencies in the workload
  - A system with very limited resources
- Choose the interface that matches your workload; POSIX is still the right choice for many programs

More Information

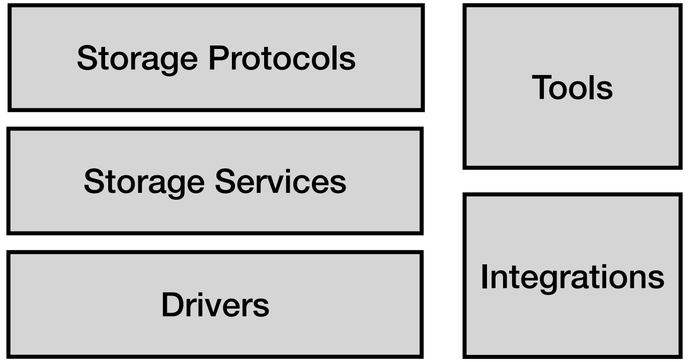
- **Efficient IO with io_uring.** J. Axboe, Linux Kernel Documentation 2019
- **Lord of the io_uring.** Shuveb Hussain, Unixism Blog io_uring guide
- **Understanding Modern Storage APIs: A systematic study of libaio, SPDK, and io_uring.** Diego Didona, Jonas Pfefferle, Nikolas Ioannou, Bernard Metzler, and Animesh Trivedi. SYSTOR'22
- **Performance Characterization of Modern Storage Stacks: POSIX I/O, libaio, SPDK, and io_uring.** Zebin Ren, Animesh Trivedi. CHEOPS '23

Questions?

SPDK

Overview

- **SPDK** is an open-source high-performance framework for storage systems using NVMe SSDs
- It performs I/O in **user-space**, eliminating all system calls and memory copies
- It **polls** device queues for completion to minimize I/O latency
- It is a modular, configurable, and customizable I/O stack

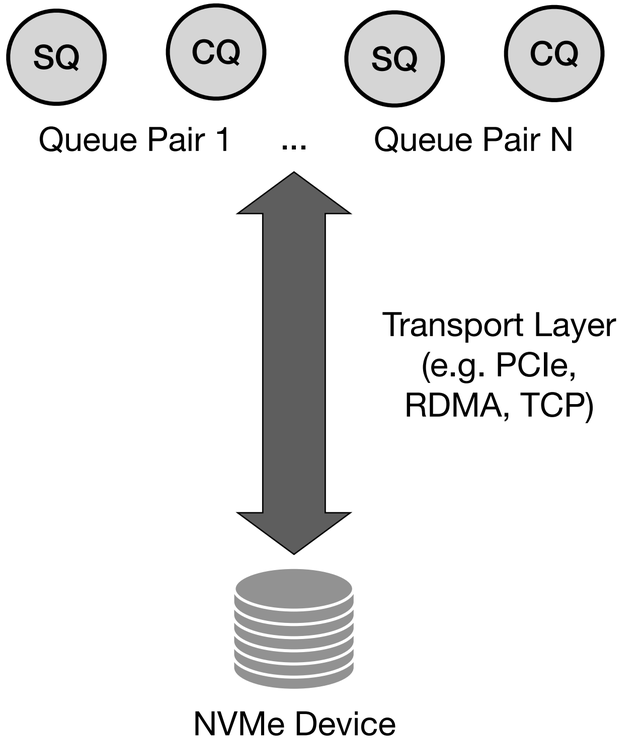


SPDK Overview

SPDK

NVMe

- **Non-Volatile Memory Express (NVMe)** is a protocol to communicate with storage devices
  - Broad feature set (e.g. NVMe-oF, NVMe ZNS, NVMe KV)
- Communication is done via **ring buffers** (submission and completion queues)
- **Multiple queue pairs can co-exist**, allowing for high data-parallelism

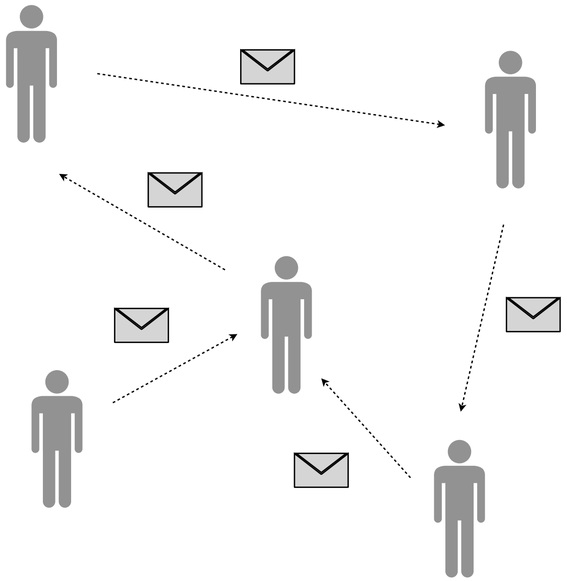


NVMe Operation

SPDK

Concurrency Model

- NVMe allows for **high data parallelism**, so we need a **programming model that can scale** accordingly
- **Shared memory concurrency is not a good fit**
  - Lock contention, poor cache performance
- The **actor programming model** is the solution
  - Each actor gets its own NVMe queue pair
  - No data sharing, no locks
  - Threads communicate via message passing

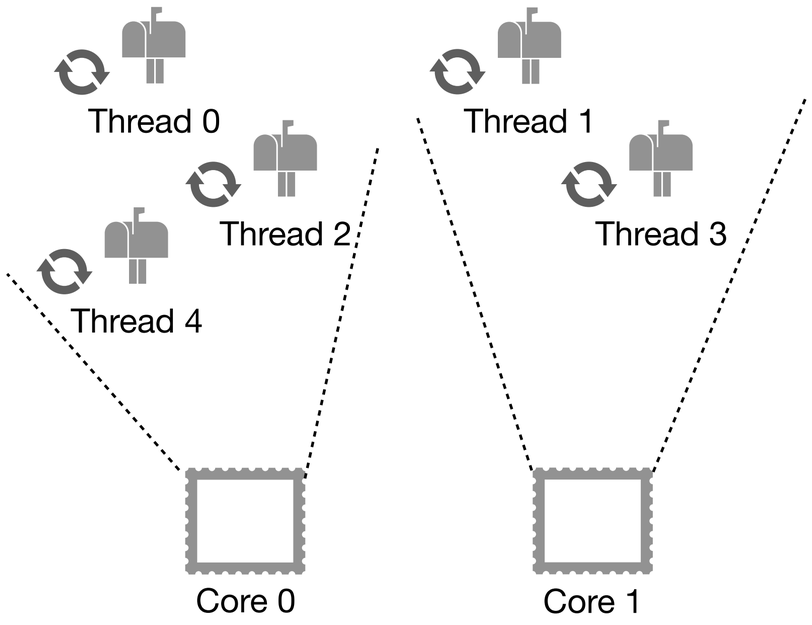


Actor programming model

SPDK

Concurrency Model

- Actors are **lightweight, user-space threads** that do not share state between them
- Threads are scheduled to run on **reactors** (i.e. CPU cores bound to the SPDK application)
- Each thread has its own message queue, which other threads use to send messages **asynchronously**
- Threads can run tasks periodically using **pollers**

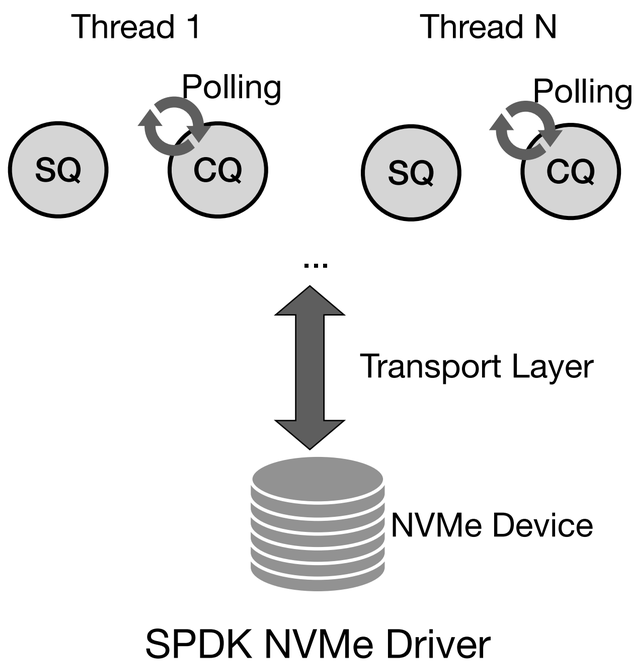


SPDK execution model

SPDK

NVMe Driver

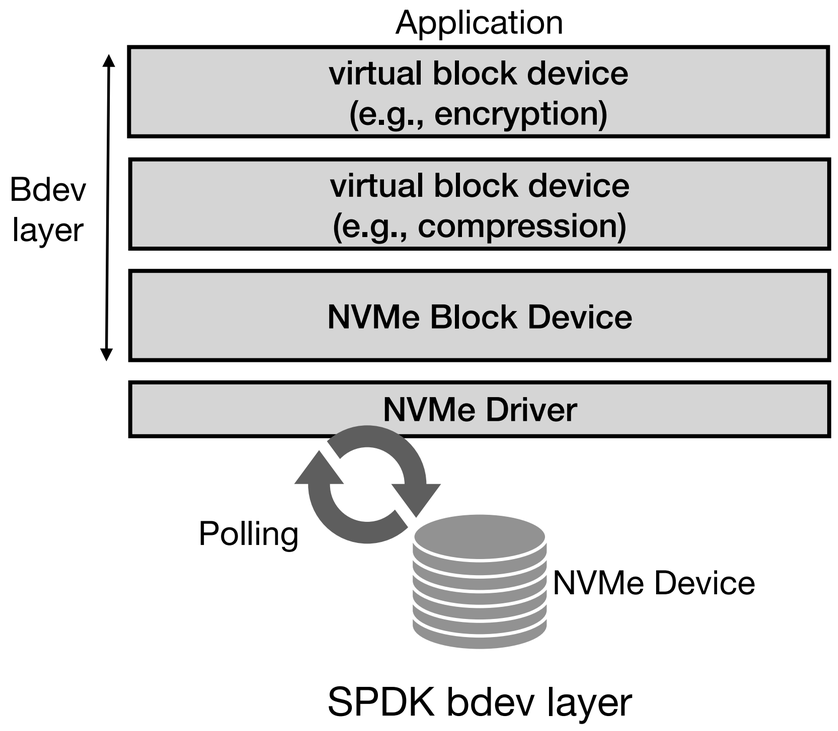
- Implemented in **user-space**, removing context switches between user-space and kernel
- Removes redundant data copies by leveraging **Direct Memory Access (DMA)**
- **Polls NVMe completion queues** to reduce latency



SPDK

Block Device Layer

- Abstracts the implementation details of the drivers, providing a **unified API** for different types of block devices
- Virtual bdevs (vbdevs) can be created and stacked on top of each other, allowing for **custom I/O stacks** without application modifications



SPDK

Block Device Layer: Example

```c
void foo_cb(struct spdk_bdev_io* bdev_io, bool success, void* cb_arg) {
    struct app_context* context = ctx;
    spdk_bdev_io_free(bdev_io);
    if(status == 0) { /* ... */}
    else { /* ... */}
}

void foo(void* ctx) {
    struct app_context* context = ctx;
    /* ... */
    spdk_bdev_read_blocks(bdev_desc, io_channel, buff, 0, 1, foo_cb, ctx);
}
```

Bdev programming example

SPDK

Other features

- SPDK implements common **storage protocols**, bundling server applications with the framework
  - NVMe-oF, iSCSI, vhost-scsi, vhost-user
- SPDK has an **acceleration framework** to leverage hardware acceleration
- SPDK has a **tracing framework** for profiling

SPDK

Considerations

- Asynchronous code is hard, in C it is even harder
- Polling wastes considerable amount of energy
- Block device not always the best fit, applications expect filesystems
- Impossible to share resources (CPU cores, SSDs) with other applications

More Information

- **SPDK Documentation.** https://spdk.io/
- **SPDK: A Development Kit to Build High Performance Storage Applications.** Ziye Yang, James R. Harris, Benjamin Walker, Daniel Verkamp, Changpeng Liu, Cunyin Chang. SC'16
- **Understanding Modern Storage APIs: A systematic study of libaio, SPDK, and io_uring.** Diego Didona, Jonas Pfefferle, Nikolas Ioannou, Bernard Metzler, and Animesh Trivedi. SYSTOR'22
- **Performance Characterization of Modern Storage Stacks: POSIX I/O, libaio, SPDK, and io_uring.** Zebin Ren, Animesh Trivedi. CHEOPS '23

Questions?